

repo_name	entry_points	spawning_patterns	async_usage	thread_usage	candidate_files_from_entrypoint	reasoning
rustdesk/rustdesk	src/main.rs,	None observed	None observed	None observed	src/server.rs, src/client.rs,	It delegates execution into UI, server, and platform-specific subsystems. No
	platform-specific				src/common.rs,	explicit async runtime or thread spawning primitives are visible here, so I
	entrypoints				src/server/connection.rs,	
	src/main.rs	Async task spawning +	Event-loop async	Dedicated worker threads	src/client/io_loop.rs, src/flutter.rs,	can't say much about concurrency model.
					src/main.rs, src/http_client.rs,	async fn main -> event loop async, tokio::spawn -> async task spawning,
					src/api/, src/db/, src/util/, src/auth/,	thread::Builder::spawn -> dedicated worker threads, worker loop: loop {
						sched.tick(); runtime.block_on(..) }. Combines Tokio async request handling
						with a scheduler thread for periodic background jobs -> hybrid runtime
dani-garcia/vaultwarden		OS thread spawning			src/mail/	coordination rather than purely async execution.
		None observed	None observed	None observed	No Rust source files from entrypoint	No Rust entrypoint exists in this repository, the startup is handled via
topjohnwu/Magisk	app/, native/					Android/native system layers, so no Rust concurrency analysis here
		OS thread spawning	No async runtime	main thread + 1 I/O thread	src/event/, src/display/, src/tty/,	Starts a winit event loop which becomes the main runtime. An IO listener
						thread is spawned for IPC/socket communication. No async runtime is used.
alacritty/alacritty	src/main.rs				src/renderer/, src/io/	Concurrency is handled via event loop + message passing between UI
		Async task spawning	Heavy async/await	Tokio worker-thread pool	helix-term/src/application.rs, helix-	thread and IO thread.
			usage (Tokio		term/src/event.rs, helix-	Uses a Tokio-based async runtime where lifecycle is driven by an async
					term/src/ui/, helix-term/src/editor/,	event loop. It constructs an application, creates an event stream, and runs
						app.run().await -> asynchronous event processing. Concurrency: Tokio
helix-editor/helix	helix-term/src/mai		runtime)		helix-core/src/, helix-view/src/	runtime -> implicitly uses a worker-thread pool for task scheduling. No
		Async task spawning,	Heavy Tokio and	Tokio worker pool, event-	src-tauri/src/services/, src-	manual thread spawning -> concurrency is likely abstracted by the async
		event-driven			tauri/src/proxy/, src-	runtime
		callbacks, background			tauri/src/database/, src-	no extra notes
					tauri/src/tray.rs, src-	
farion1231/cc-switch	src-tauri/src/lib.rs	worker loops	Tauri async runtime	driven threads	tauri/src/commands/, src-	
	src/main.rs	Worker pool? (through Config),	No explicit async	parallel filesystem traversal	tauri/src/settings.rs, src-	
		shared command		using configurable thread	tauri/src/lightweight.rs	
sharkdp/fd		state via Arc	runtime	count	src/main.rs, src/walk.rs,	Config carries a thread count and passes it into walk::scan. Uses Arc for
	src/main.rs	config setup, CLI		Some	src/filesystem.rs, src/filter.rs	shared command execution state -> worker threads?. No async runtime
		parsing, ctrlc setup,			command.rs, signals.rs	initialises engine state, loads configuration, routes execution into different
		execution mode				
nushell/nushell		dispatch (REPL/script/LSP/MCP),	high			modes (interactive shell, script runner, language server, MCP server)
		plugin loading				This coordinates editor state, tabs, documents, navigation, and updates, UI
	lapce-	Background spawning	async/event-driven	Mostly single-thread (Rc,	main_split.rs, window_tab.rs,	
		through proxy				
		callbacks and reactive	structure, proxy	RwSignal, Memo),	editor.rs, doc.rs, proxy.rs,	
	app/src/main_spli	event actions	communication and	background worker threads	keypress.rs,	
		(create_ext_action,	reactive		panel/implementation_view.rs,	
lapce/lapce	t.rs	proxy request				
		handlers)	effects/signals	elsewhere	editor_tab.rs	state management and concurrent backend services
	yazi-	tokio::main + staged	async via init()	layered services +	dispatcher, executor, watcher,	Initialises multiple subsystems which likely spawn their own async tasks,
sxyazi/yazi	fm/src/main.rs	init + service startup	pipeline + serve()	futures	runner, plugin, dds	
		chain				

	src/firecracker/sr	Creates microVMs: vmm::builder::build_and_boot_microvm(). Spawns vCPU threads and API/VMM runtime components during VM startup	Uses an EventManager to drive VM devices, API events, timers, I/O	Main thread performs initialisation, API server thread handles requests, VMM thread manages VM state, one thread per guest vCPU is created by KVM/VM startup logic	src/firecracker/src/api_server_adapter.rs, src/firecracker/src/api_server.rs, src/vmm/src/builder.rs, src/vmm/src/lib.rs, src/vmm/src/resources/ src/vmm/src/device_manager/ src/jailer/src/main.rs	main_exec() performs setup and then branches into API mode or VM startup. Then build_and_boot_microvm()
firecracker-microvm/firecracker	c/main.rs	Spawns external shell processes/commands when executing recipes defined in a justfile	No async usage in entry point	No thread creation visible in entry point	src/lib.rs, src/config.rs, src/justfile.rs, parser/evaluator modules, recipe execution modules, shell/command execution modules	Completion setup, CLI initialisation, delegates application logic to just::run()
casey/just	src/main.rs					
surrealdb/surrealdb						
qdrant/qdrant						
idx/mise						
rustfs/rustfs						
cloudflare/pingora						
neondatabase/neon						
tursodatabase/turso						
ast-grep/ast-grep						